
Computer Graphics Report

Jiachen Lyu
5613244

University of Warwick

December 19, 2025

1 Introduction

This project implemented a basic 3D FPS game using DirectX 12 and C++, featuring fundamental lighting, collision detection, materials, and animation systems. Instanced rendering was employed to support large-scale scenes and improve rendering efficiency. The models were provided in GEM format, and the stb image library was used for loading texture maps.

The project completed the setup of a standard rendering pipeline and implemented multiple vertex and pixel shaders to meet different rendering requirements. The project extends the base rendering framework provided in the course by introducing higher-level abstractions to improve code extensibility. In addition, a data-driven approach was adopted, with scene data loaded from CSV files to construct the game world.

2 Technical Implementation

2.1 Mesh Rendering

Considering that the same mesh may be reused multiple times within the project, a MeshManager class was implemented to manage mesh resources. Each mesh is loaded only once and reused when required, thereby reducing unnecessary GPU memory usage.

```
1 void loadmesh(Core* core, vector<vector<  
    ANIMATED_VERTEX>> vertices, vector<vector<  
    unsigned int>> indices, string meshname) {  
2     if (meshesmanager.find(meshname) !=  
        meshesmanager.end()) {  
3     }  
4     else {  
5         vector<GeneralMesh*> meshes;  
6         for (int i = 0; i < vertices.size(); i++) {  
7             GeneralMesh* mesh = new GeneralMesh();  
8             mesh->init(core, vertices[i], indices[i]);  
9             meshes.push_back(mesh);  
10        }  
11    }
```

```
12     meshesmanager.insert({ meshname, meshes });  
13 }  
14  
15  
16 }
```

Listing 1: Load Meshes function

During model loading, each model only stores its mesh data without immediately initializing the mesh objects. After loading is complete, the mesh data is passed to the MeshManager class together with an associated mesh name. If a mesh with the given name does not already exist, the mesh is initialised using the provided data and stored in the MeshManager. When required, meshes are retrieved from the manager by name and used for draw calls.

2.1.1 Data-Driven

The project uses CSV files to store map data, including the distribution of all static and dynamic models in the scene. Three separate CSV files are used for this purpose. The first file stores information for general game objects, including their positions, the GEM models used, and the corresponding identifiers for the custom MeshManager and TextureManager. The second file stores data related to game objects rendered using instancing, while the third file records all position coordinates of instanced objects. After instancing was introduced into the project, the file for general game objects was primarily used to store data for animated objects only.

```
1 vector<Objectload> LoadData(vector<Objectload>*&  
    scv1, vector<Objectload>*& scv2) {  
2     ifstream file("../Resources/loadfile1.csv");  
3  
4     string line;  
5  
6     getline(file, line);  
7     Objectload getinformation;  
8     while (getline(file, line)) {
```

```

9      std::stringstream theline(line);
10     std::string word;
11
12     getline(theline, word, ',');
13     getinformation.type = word;
14     getline(theline, word, ',');
15     getinformation.name = word;
16     getline(theline, word, ',');
17     getinformation.position.x = stof(word);
18     getline(theline, word, ',');
19     getinformation.position.y = stof(word);
20     getline(theline, word, ',');
21     getinformation.position.z = stof(word);
22     getline(theline, word, ',');
23     getinformation.rotation.x = stof(word);
24     getline(theline, word, ',');
25     getinformation.rotation.y = stof(word);
26     getline(theline, word, ',');
27     getinformation.rotation.z = stof(word);
28     getline(theline, word, ',');
29     getinformation.location = (word);
30     getline(theline, word, ',');
31     getinformation.textureName = (word);
32     getline(theline, word, ',');
33     getinformation.iscollider = stoi(word);
34     getline(theline, word, ',');
35     getinformation.meshname = (word);
36     if (getinformation.type == "static") {
37         scv1->push_back(getinformation);
38     }
39     else {
40         scv2->push_back(getinformation);
41     }
42 }
43
44 file.close();
45 return scv;
46 }

```

Listing 2: Load CSV Method

2.1.2 Mesh Rendering and Alpha Testing

During mesh rendering, the vertices of a mesh are multiplied by the World matrix and the View-Projection matrix to transform them from model space to world space and then into clip space. The rendering pipeline subsequently performs clipping, after which the final fragments to be drawn are determined. During pixel rendering, alpha testing is applied to determine whether a pixel should be rendered based on its alpha value, thereby enabling the correct rendering of opaque meshes. After textures are introduced, the required pixel colour is obtained by sampling the texture according to the corresponding UV coordinates.

```

1 PS_INPUT VS(VS_INPUT input)
2 {
3     PS_INPUT output;
4     output.Pos = mul(input.Pos, W);
5     output.Pos = mul(output.Pos, VP);
6     output.Normal = mul(input.Normal, W);
7     output.Tangent = mul(input.Tangent, W);
8     output.TexCoords = input.TexCoords;
9     return output;
10 }
11 // The basic vertex transformation process

```

Listing 3: From Vertex to Pixel



Figure 1: Mesh Rendering and Alpha Testing

In the pixel shader stage, alpha testing is applied to models such as leaves and grass to discard transparent fragments. The figure above shows the result of mesh rendering, including opaque tree trunks, the T-Rex model, and tree leaves that have been discarded through alpha testing.

2.1.3 Vertex Animation

In the vertex shader, vertex offsets are applied to simulate the swaying motion of grass. First, an offset parameter is calculated based on the current time and wind intensity. Next, a weighting factor is derived from the UV coordinates, where vertices closer to the root receive a smaller offset and those closer to the tip receive a larger offset. However, because the material used does not represent a single blade of grass from bottom to top, but instead contains multiple grass textures within the same material, the UV coordinates do not consistently correspond to the root-to-tip direction of individual blades. As a result, this weighting effect is not very pronounced. The final offset is then computed by applying periodic oscillations to the x and z directions. A spatial phase offset is introduced to prevent all grass instances from moving synchronously. The resulting displacement is multiplied by the UV-based weight and the wind intensity to produce the final vertex offset.

```

1 float3 GetWindEffect(float3 VertexWorldPos,
2     float GrassUV, float WindSpeed, float
3     WindStreth, float GameTime)
4 {
5     float Timer = GameTime * WindSpeed;
6
7     float3 GrassWindOffset = float3(0, 0, 0);
8
9     float UVChange = 1 - abs(GrassUV);
10    //Due to the issue with the texture, the UV
11    //weights did not achieve a satisfactory
12    //result.
13    GrassWindOffset.z = UVChange * sin(cos(
14    Timer + VertexWorldPos.z)) * WindStreth;
15    GrassWindOffset.x = UVChange * cos(sin(
16    Timer + VertexWorldPos.x)) * WindStreth;
17    //Introduce world coordinates for pseudo-
18    //randomness
19    GrassWindOffset.y = 0;
20    return GrassWindOffset;

```

15 }

Listing 4: Wind implement

2.1.4 Instancing of Meshes

To reduce CPU overhead and improve GPU utilization, instancing was introduced to render a large number of similar objects, such as grass and trees. During initialization, an instance buffer is created on the GPU to store per-instance data, including the world transformation matrix of each object. During rendering, a single draw call is issued to render multiple instances of the same mesh, with the GPU reading the corresponding transformation matrices from the instance buffer to render objects at different positions. Since instanced rendering uses the same rendering pipeline and shared global constant buffers, this approach is most suitable for objects that share the same mesh and have minimal per-instance variation.



Figure 2: Instancing with Grass

2.2 Basic Camera Control System

The camera control system consists of two main components: camera orientation control and camera movement control. Camera orientation is controlled using mouse input, while camera movement is handled through the keyboard using the W, A, S, and D keys.

Each frame, the mouse displacement along the x and y axes is captured and used to update the camera's yaw and pitch angles. Based on these values, the camera's forward direction vector is computed using the corresponding mathematical formulas, as illustrated in the code. Once the forward direction is obtained, it is added to the camera position to calculate the target "to" vector. The right direction is then derived using the cross product between the forward vector and the world up direction. Finally, a LookAt matrix is constructed to update the camera orientation.

During rotation, limits are applied to the pitch angle to prevent the camera from flipping.

```
1 forward.x = cosf(pitch) * cosf(yaw);
2 forward.y = sinf(pitch);
3 forward.z = cosf(pitch) * sinf(yaw);
```

Listing 5: Convert the Angle to Forward Vector

Camera movement is performed relative to the current viewing direction, with the W, S, A, and D keys. Each frame, keyboard input is processed to determine the movement direction vector based on the camera's forward and right vectors. This direction vector is multiplied by the movement speed and the frame time to obtain the final displacement, which is then added to the current camera position to produce the updated position. Since the camera is constrained to move on the ground plane and does not move along the y-axis, the x and z components of the forward and right vectors are normalized independently before computing the movement. This prevents the movement speed from decreasing as the camera's viewing direction deviates further from the horizontal plane.

```
1 Vec4 groundright = Vec4(right.x, 0, right.z, 0).
  normalize();
2 Vec4 groundforawrd = Vec4(forward.x, 0, forward.
  z, 0).normalize();
3 Vec4 delta(0, 0, 0, 0);
4 if (win.keys['A']) delta += Vec4(groundright.x,
  0, groundright.z, 0);
5 if (win.keys['D']) delta -= Vec4(groundright.x,
  0, groundright.z, 0);
6 if (win.keys['W']) delta += Vec4(groundforawrd.x
  , 0, groundforawrd.z, 0);
7 if (win.keys['S']) delta -= Vec4(groundforawrd.x
  , 0, groundforawrd.z, 0);
8 //delta=delta.normalize();
9 delta *= cameramovespeed * rexdt;
10 from = from + delta;
```

Listing 6: Move Control

2.3 Animation Control

The project uses skeletal animation. Data such as the position and rotation of each bone is stored, and smooth motion is achieved over time through interpolation. The resulting data is then uploaded to the shader, where the final animation calculations are performed.

This project uses two animated models, the T-Rex and the UZI, each with its own animation control logic. The animation system provided in the course supports animation switching and looping, and this project builds upon it to implement gameplay-driven animation control.

For the T-Rex, three animations are used: attack, run, and death. Each frame, the T-Rex's state is evaluated to select the appropriate animation. If its health drops below zero, the death animation is triggered and played until completion, after which the T-Rex is respawned. If the T-Rex is alive, a collision with the player triggers the attack animation; otherwise, the run animation is played. Damage is applied after each completed attack animation.

```
1 void anim(float rexdt, string status, Vec3 from)
  {
2   if (status == "death") {
3
4     if (status != Animatestatus) {
5       Animatestatus = status;
6       isdying = true;
```

```

7         dieanim = 0;
8     }
9     else {
10         if (enemymodelinstance.animationFinished
11             ) == true || dieanim > 2.8999f) {
12             health = 100.f;
13             Animatestatus = "run";
14
15             die(from);
16             isdying = false;
17             dieanim = 0;
18         }
19         else {
20             enemymodelinstance.update("death",
21                 rexdtd);
22             dieanim += rexdtd;
23         }
24     }
25
26 }
27
28 }
29
30 else {
31     Animatestatus = status;
32     enemymodelinstance.updatewithControl(
33         Animatestatus, rexdtd);
34     if (enemymodelinstance.animationFinished())
35     {
36         enemymodelinstance.resetAnimationTime();
37         attatcktime = 0;
38     }
39 }
40
41 };

```

Listing 7: TRex Animation Control Logic

The control logic of the UZI is relatively complex and involves a total of five different animations. The weapon supports aiming and firing through the use of the right and left mouse buttons, and includes a reloading function. Reloading can be triggered either manually by pressing the R key or automatically when the ammunition count reaches zero. Once the reload animation begins, no other animations can be played until the reload animation has fully completed.

When the right mouse button is pressed, the weapon switches to the aiming animation. While in this state, pressing the left mouse button triggers a check to determine whether shooting is allowed, including validation of the remaining ammunition and the firing cooldown. The cooldown duration is defined as the full playback time of a single firing animation. If shooting is permitted, the weapon switches to the aimed firing animation. While the firing animation is active, it continues to loop unless the left mouse button is released or other conditions required for shooting are no longer satisfied.

If only the left mouse button is pressed, the weapon performs non-aimed firing, which follows logic similar to that of aimed firing.

```

1 void anim(Vec3 from, vector<Enemies*> enemies,
2     bool mouse[3], float rexdtd, Vec3 forawrd,
3     bool changeR, vector<Fire*> fires) {

```

```

2     if (changeR&&!ischange) {
3         ischange = true;
4         heromodelinstance.update("17 reload", rexdtd);
5         //start reload
6     }
7     if (bullet <= 0 && !ischange) {
8         ischange = true;
9         heromodelinstance.update("17 reload", rexdtd);
10        //auto reload
11    }
12    if (ischange) {
13        heromodelinstance.update("17 reload", rexdtd);
14        if (heromodelinstance.animationFinished()) {
15            heromodelinstance.resetAnimationTime();
16            ischange = false;
17            bullet = 30;
18            shot = true;
19        }
20    }
21    else {
22        if (!shot) {
23            timecount += rexdtd;
24            if (timecount > cooldown) {
25                shot = true;
26                timecount = 0.f;
27            }
28        }
29    }
30
31    if (mouse[1]) {
32        //zoom
33        if (mouse[0] && bullet > 0) {
34            if (shot) {
35                // zoom shot
36                raytest.init(from+forward*3, forawrd);
37                //plus 3 to make bullet shot front
38                of camera
39                for (int i = 0; i < enemies.size(); i
40                    ++){
41                    float t;
42                    if (enemies[i]->enemymodel.collision
43                        .rayAABB(raytest, t)) {
44                        //hit the target or not
45                        enemies[i]->health -= 5;
46                        for(int i=0;i< fires.size();i++){
47                            if(!fires[i]->active){
48                                fires[i]->work(rexdtd, from +
49                                    (forawrd * (t + 3)));
50                                //particle effect
51
52                                break;
53                            }
54                        }
55                        if (enemies[i]->health <= 0&&!
56                            enemies[i]->isdying) {
57                            score += 5;
58                            enemies[i]->isdying = true;
59                        }
60                        break;
61                    }
62                }
63                bullet--;
64                shot = false;
65            }
66            heromodelinstance.update("13 zoom fire",
67                rexdtd);
68
69            if (heromodelinstance.animationFinished()
70                ) {
71                heromodelinstance.resetAnimationTime();

```



```

69         shot = true;
70     }
71 }
72 else {
73     heromodelinstance.updatewithControl("11
zoom idle", rexdt);
74 }
75 }
76 }
77 }
78 else if (mouse[0] && bullet > 0) {
79     //normal shot
80     if (shot) {
81         raytest.init(from + forward * 3, forawrd
);
82         for (int i = 0; i < enemies.size(); i++)
83         {
84             float t;
85             if (enemies[i]->enemymodel.collision.
rayAABB(raytest, t)) {
86                 enemies[i]->health -= 5;
87                 for (int i = 0; i < fires.size(); i
++) {
88                     if (!fires[i]->active) {
89                         fires[i]->work(rexdt, from + (
forawrd * (t+3)));
90
91                     break;
92                 }
93             }
94         }
95         if (enemies[i]->health <= 0 && !
enemies[i]->isdying) {
96             score += 5;
97             enemies[i]->isdying = true;
98         }
99         break;
100     }
101 }
102 }
103 bullet--;
104 shot = false;
105 }
106 heromodelinstance.update("08 fire", rexdt);
107 if (heromodelinstance.animationFinished())
108 {
109     heromodelinstance.resetAnimationTime();
110     shot = true;
111 }
112 else {
113     heromodelinstance.updatewithControl("04
idle", rexdt);
114     //idle
115 }
116 }
117 }

```

Listing 8: UZI Animation Control Logic

2.4 Collision System

This project employs both AABB collision detection and ray-based collision detection. Enemy T-Rex objects use a minimum bounding box as their collision box.

To generate the minimum bounding box, all vertices of the model are traversed to obtain the maximum and minimum values along the x, y, and z axes, forming an AABB in model space. This bounding box is then

transformed into world space based on the object's position to obtain the final collision box.

Collision is determined by checking whether two bounding boxes overlap along each axis. If, in any direction, one bounding box lies completely on one side of the other, then they cannot intersect. If this does not occur on any axis, the two bounding boxes are considered to be colliding.

```

1 bool AABBtest(const Vec3& minB, const Vec3& maxB
)
2 {
3
4     if (realmaxpoint.x < minB.x || realminpoint.x
> maxB.x)
5         return false;
6     if (realmaxpoint.y < minB.y || realminpoint.y
> maxB.y)
7         return false;
8     if (realmaxpoint.z < minB.z || realminpoint.z
> maxB.z)
9         return false;
10
11     return true;
12 }

```

Listing 9: Collision Test

To handle rotation, the eight corner vertices of the bounding box are first constructed from the minimum and maximum points. All vertices are then rotated, and new minimum and maximum values are recalculated from the transformed vertices. The correct update order is to first compute the rotated minimum and maximum points in model space using the rotation matrix, and then add the object's position to obtain the final collision box in world space.

```

1 void update(Vec3 position, Matrix scv) {
2     //rotation need calculate the 8 corner point
and find the new min and max point
3     vector<Vec3>corner;
4     corner.push_back(Vec3(minpoint.x, minpoint.y,
minpoint.z));
5     corner.push_back(Vec3(minpoint.x, minpoint.y,
maxpoint.z));
6     corner.push_back(Vec3(minpoint.x, maxpoint.y,
minpoint.z));
7     corner.push_back(Vec3(minpoint.x, maxpoint.y,
maxpoint.z));
8     corner.push_back(Vec3(maxpoint.x, minpoint.y,
minpoint.z));
9     corner.push_back(Vec3(maxpoint.x, minpoint.y,
maxpoint.z));
10    corner.push_back(Vec3(maxpoint.x, maxpoint.y,
minpoint.z));
11    corner.push_back(Vec3(maxpoint.x, maxpoint.y,
maxpoint.z));
12    for (int i = 0; i < 8; i++) {
13        corner[i] = scv.MulVec3(scv, corner[i]);
14    }
15    realminpoint = Vec3(11111, 11111, 11111);
16    realmaxpoint = Vec3(-11111, -11111, -11111);
17    for (int i = 0; i < 8; i++) {
18        if (realmaxpoint.x < corner[i].x) {
19            realmaxpoint.x = corner[i].x;
20        }
21        if (realmaxpoint.y < corner[i].y) {
22            realmaxpoint.y = corner[i].y;
23        }
24        if (realmaxpoint.z < corner[i].z) {

```

```

25     realmaxpoint.z = corner[i].z;
26 }
27 if (realminpoint.x > corner[i].x) {
28     realminpoint.x = corner[i].x;
29 }
30 if (realminpoint.y > corner[i].y) {
31     realminpoint.y = corner[i].y;
32 }
33 if (realminpoint.z > corner[i].z) {
34     realminpoint.z = corner[i].z;
35 }
36 }
37 realminpoint += position;
38 realmaxpoint += position;
39
40
41 }

```

Listing 10: rotation collider box

The project uses ray casting to determine whether a shot hits an object. For ray-box intersection, the entry and exit times of the ray are computed separately along each axis. If an object lies behind the ray, the computed entry and exit times will be negative, in which case the entry and exit values need to be reversed; this is handled automatically using value comparisons.

The maximum entry time and the minimum exit time are then selected as the actual entry and exit times of the ray. A collision is determined by comparing these times: if the exit time occurs before the entry time, no collision is detected. The variable *t* represents the time at which the ray enters the bounding box.

Objects located behind the ray are treated as non-colliding, as shooting in the project is directional and cannot hit objects located behind the firing direction

```

1 bool rayAABB(const Ray& r, float& t)
2 {
3     Vec3 s = (realminpoint - r.o) * r.invdir;
4     Vec3 l = (realmaxpoint - r.o) * r.invdir;
5     //calculate the time of input and output in
      xyz directions
6     Vec3 s1 = Min(s, l);
7     Vec3 l1 = Max(s, l);
8     //if the direction is negative, swap the value
9     float ts = max(s1.x, max(s1.y, s1.z));
10    //the max number in min, the real enter time
11    float tl = min(l1.x, min(l1.y, l1.z));
12    //the min number in max, the real leave time
13    t = min(ts, tl);
14
15
16    if (tl < 0.0f)
17        return false;
18    //direction is negative
19
20
21    return (ts < tl);
22
23
24 }

```

Listing 11: Ray Test

2.5 Normal Mapping, Lighting, and Textures

During model loading, UV coordinates are read from the model data. For custom geometry, UV coordinates can also be manually defined when creating the vertices. In the pixel shader, the UV coordinates are used to sample the texture and obtain the corresponding colour, which is then used for rendering.

To improve efficiency, a texture manager class was implemented. This class uses a map to store both colour textures and normal maps. Two initialization approaches are supported. For GEM models, texture paths are read during model loading and passed to the texture manager for initialization. For non-GEM objects, such as the skybox and UI elements, textures are loaded by file path during the game initialization stage.

```

1 void load(Core* core, vector<string> name, vector
      <string>NHname, string texturename)
2 {
3     if (textures.find(texturename) != textures.end
        ()) {
4
5     }
6     else {
7         vector <Texture*> scv;
8         vector <Texture*> scv1;
9         for (int i = 0; i < name.size(); i++) {
10
11
12             Texture* texture = new Texture;
13
14             texture->load(core, name[i]);
15             scv.push_back(texture);
16
17
18             Texture* texture1 = new Texture;
19
20             texture1->load(core, NHname[i]);
21             scv1.push_back(texture1);
22
23         }
24         textures.insert({ texturename, scv });
25         NHtextures.insert({ texturename, scv1 });
26     }
27
28
29 }

```

Listing 12: Load Textures Function

The project implements directional lighting. The light direction and strength are passed to the pixel shader through the pixel shader constant buffer. The light direction is first normalized and inverted to obtain the direction from the surface point to the light source. The dot product between the surface normal and the light direction is then computed to obtain the cosine of the angle between them. If this value is less than zero, the surface receives no light. The result is multiplied by the light strength to obtain the final brightness of the pixel.

A basic ambient lighting term was also introduced to ensure a minimum level of illumination.

```

1 float3 ComputeDirectionalLight(Light L, float3
    normal )
2 {
3     float3 lightVec = -normalize(L.Direction); //
    calculate the vector from surface to light
4     float ndotl = max(dot(lightVec, normal), 0.0
    f); //Lambert Cosine Law, calculate the
    strength of light
5     float3 lightStrength = L.Strength * ndotl; //
    mix
6
7     return lightStrength;
8 }

```

Listing 13: Light Function

To achieve more accurate lighting results, normal mapping is used in this project. After sampling the normal map using the UV coordinates, the sampled normal is transformed using the TBN basis to obtain a world-space normal, which is then used in the lighting calculations.

```

1 float4 PS(PS_INPUT input) : SV_Target0
2 {
3
4     float4 colour = tex.Sample(samplerLinear,
    input.TexCoords);
5     if (colour.a < 0.5f)
6         discard;
7     float3 normalTS = NHtex.Sample(samplerLinear
    , input.TexCoords).xyz;
8     normalTS = normalize(normalTS * 2.0f - 1.0f)
    ; // turn Vec to normal
9
10
11     float3 N = normalize(input.Normal);
12     float3 T = normalize(input.Tangent);
13     T = normalize(T - dot(T, N) * N);
14     float3 B = cross(N, T);
15     float3x3 TBN = float3x3(T, B, N);
16     float3 normalWS = normalize(mul(normalTS,
    TBN)); //turn to world space normal
17
18     Light L;
19     L.Strength = Strength;
20     L.Direction = normalize(Direction);
21
22     float4 ambient = gAmbientLight ; //ambient
    light
23
24     float4 directLight = ComputeLighting(L,
    input.PosW, normalWS);
25     float3 lighting = ambient.rgb + directLight.
    rgb;
26     float3 finalColor = colour.rgb * lighting;
27     return float4(finalColor, colour.a);
28 }

```

Listing 14: Pixel Shader for Lighting

2.6 Other Effects

2.6.1 Skybox

In this project, the skybox effect is implemented using a custom spherical mesh combined with a skybox texture. During rendering, the skybox participates in depth testing but does not write to the depth buffer, ensuring that it always appears as a background element and does not occlude other objects in the scene.

The position of the skybox is updated to follow the camera position, creating the visual effect of an infinitely distant sky.



Figure 3: skybox

2.6.2 UI

The project implements several basic UI elements. These UI elements do not participate in depth testing and are rendered last, ensuring that they are never occluded by other objects in the scene. Numeric information is rendered using a bitmap font and is updated in real time. The bullet icon in the lower-left corner is updated dynamically based on the current remaining ammunition count.

The UI is essentially implemented using planes with textures. The vertex positions of these planes represent pixel positions on a 1920×1080 screen. During rendering, the vertex coordinates are transformed directly into screen space rather than undergoing the standard sequence of spatial transformations.

```

1 PS_INPUT VS(VS_INPUT input)
2 {
3     PS_INPUT output;
4
5
6     output.Pos = float4((input.Pos.x-960)/960, (
    input.Pos.y-540)/540, 0.0f, 1.0f);
7     //calculate to [-1,1]
8
9     output.Normal = input.Normal;
10    output.Tangent = input.Tangent;
11
12    output.TexCoords = input.TexCoords;
13
14    return output;
15 }

```

Listing 15: Vertex Shader for UI

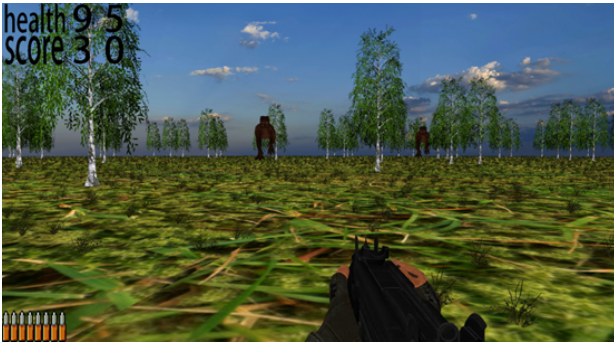


Figure 4: UI

2.6.3 Particles Effect

This project implements a spark-like particle effect. After a successful ray intersection is detected, a burst of spark particles is rendered at the position of the hit enemy.

The particle effect is rendered on a billboard, with the center of the plane representing the explosion origin. Pseudo-random vectors are generated in three directions and normalized to define the movement direction of each particle, while particle speeds are also determined using pseudo-random values. Each particle's position is updated over time, and in the pixel shader the distance between the particle and the current pixel is computed. The pixel brightness is then smoothly interpolated based on this distance.

Each particle effect consists of 109 particles. The shader iterates over all particles and accumulates their colour contributions to produce the final pixel colour.

To ensure consistent and stable particle motion within a single particle effect, pseudo-random values are generated based on position information; otherwise, the particle behaviour would appear completely chaotic.



Figure 5: Particle Effect

```

1 float3 FireworkParticles( float3 worldPos, float3
    fireworkPos, float time, float3 baseColor
2 )
3 {
4     float seed = hash1(dot(fireworkPos, float3
        (12.9898, 78.233, 45.164)));
5     //random seed
6     float3 color = float3(0, 0, 0);

```

```

7
8     const int NUM_PARTICLES = 109;
9
10    for (int i = 0; i < NUM_PARTICLES; i++)
11    {
12
13        float speed = lerp(3.0, 6.0, hash1(i *
            seed * 31.35));
14        //get random speed form 3 to 6
15        float3 dir = normalize(float3(
            hash1(i*seed * 33.95)*2-1 ,
16            hash1(i*seed * 7.1)*2-1 ,
17            hash1(i*seed * 25.12)*2-1
18        ));
19        //random direction
20        float3 p = fireworkPos + dir * speed *
            time;
21        //now position of particle
22        float dist = length(worldPos - p);
23        float size = 0.1;
24        float fade = saturate(1.0 - time);
25
26        float intensity = smoothstep(size, 0.0,
            dist) * fade;
27        //calculate color based on distance
28        //the dis more close to zero more light
29
30        color += baseColor * intensity;
31    }
32
33    return color;
34 }
35

```

Listing 16: Particle Effect implement

3 Performance Analysis

This project uses tree models for performance testing. Performance is evaluated by comparing the frame rates achieved using the standard rendering approach and the instanced rendering approach under the same number of rendered objects. This comparison highlights the impact of CPU performance bottlenecks caused by multiple draw calls on overall frame rate.

In the test scene, directional lighting is applied to the trees. In addition, the scene also includes a skybox. The tests were conducted on a system equipped with an NVIDIA GeForce RTX 5070 Laptop GPU (8 GB) and an AMD Ryzen 9 8945HX with Radeon Graphics.



Figure 6: test scene

Table 1: Performance Test Results Under Different Conditions

Render Method	Tree Number	FPS
Normal	100	337
Normal	200	195
Normal	300	133
Normal	400	101
Normal	500	81
Normal	600	61
Normal	800	47
Normal	1000	39
Instancing	100	2747
Instancing	200	1653
Instancing	300	1389
Instancing	400	940
Instancing	500	745
Instancing	600	653
Instancing	800	514
Instancing	1000	412

The table clearly shows that when using the standard rendering approach, CPU performance becomes a significant bottleneck, severely limiting GPU utilization and resulting in a performance gap of nearly an order of magnitude. This test was conducted using tree models with relatively high vertex counts but a limited number of instances.

If the trees were replaced with grass, which requires rendering a much larger number of instances, the number of draw calls would increase substantially. Without instancing, GPU utilization would become extremely low, making the application difficult to run smoothly. In practice, this project renders approximately 50,000 grass instances using instanced rendering within a spatial range of -125 to 125. Based on this analysis, it can be reasonably inferred that rendering such a scene using standard, non-instanced rendering would not be feasible while maintaining acceptable performance.

4 Limitations

The overall structure of the project is relatively disorganized, as it evolved gradually from daily practice exercises without an initial architectural design. Although a structural refactor was carried out during the middle stage of development, the final structure resulted in a tightly nested dependency hierarchy, where classes depend on each other in multiple layers. As a result, adding new features requires careful consideration of dependency relationships, which makes further development more difficult.

The UZI model exhibits noticeable texture jittering. This issue disappears when depth testing is disabled, indicating that it is likely caused by z-fighting. One possible cause is that multiple meshes within the model are positioned too closely to each other.

5 Future Work

Undoubtedly, the project needs a better overall structure, not only in the code. Since the project uses many external files, clear rules for file paths are needed. Random placement of shaders, assets, and CSV files increases the chance of errors.

The project uses separate classes for different object types. Many of these classes have similar functionality but are not clearly related. Adding a proper inheritance structure can reduce duplicated code and improve maintainability.

In the texture management system, textures are stored in groups using a map, which can lead to redundant storage when different groups share the same textures. A better approach would be to store each texture only once and reference them by index.

6 Conclusion

This project implemented an FPS game using C++ and DirectX 12. The game includes interaction logic, animation switching, textures, a skybox, particle effects, instancing, lighting with normal mapping, vertex animation, and other related features. Through the use of instancing, the project is able to render a large number of grass objects while maintaining smooth performance. Spark-like particle effects are used to simulate impact effects when a target is hit, and reasonable animation transition logic is implemented for both the player character and enemies.

Although there is still room for improvement, the project has a relatively high level of completion. During development, the project also increased my understanding of the rendering pipeline, which will be beneficial for future game development. Overall, the project achieved its intended objectives.

Project Repository

The source code for this project is available on GitHub: <https://github.com/AODINGLVP/newworld.git>